

## NOTES

# Regular Expressions (Regex) & Essential Math Functions



## ✓ Regular Expressions and Mathematical Operations in Python

### ✓ Introduction to Regular Expressions

Regular expressions (regex) are powerful tools for searching, matching, and manipulating text. Regex helps find specific text patterns within strings, which is useful for data extraction, validation, and formatting tasks.

Regex can automate text-processing tasks like:

**Data extraction:** Extracting emails, phone numbers, or other data from unstructured text.

**Data validation:** Ensuring text adheres to formats like dates or emails.

**String manipulation:** Replacing or reformatting text.

Python's `re` module provides several functions for regex operations, such as `re.search()`, `re.match()`, `re.findall()`, etc.

```
# Import the regex module
import re
```

### Basic Regex Patterns

#### ✓ 1. Basic Characters

Regex supports simple character matching, where you search for exact text patterns.

##### ✓ Literal Matching

Finds exact text in the string. Literal matching is straightforward and useful for finding specific words or phrases.

Use `re.search(pattern, text)` to find an exact match within a string.

```
import re

# Example string
text = "Hello, World!"

# Define the pattern to search
pattern = "World"

# Perform the search
match = re.search(pattern, text)

# Output the result
if match:
    print("Match found:", match.group()) # Expected Output: "World"
else:
    print("No match found")

➦ Match found: World
```

`re.search()` looks for "World" in text. If found, it returns the matched text with `match.group()`. This search is case-sensitive.

##### ✓ Case Sensitivity

By default, regex is case-sensitive, meaning "world" and "World" are different. Sometimes you want to match text regardless of case (e.g., matching "HELLO" or "Hello").

Use `re.IGNORECASE` to ignore case differences.

```
pattern = re.compile("world", re.IGNORECASE)
match = pattern.search(text)

# Output the result
if match:
    print("Case-insensitive match found:", match.group()) # Expected Output: "World"
else:
    print("No match found")

➦ Case-insensitive match found: World
```

Here, `re.compile("world", re.IGNORECASE)` makes the pattern case-insensitive. The pattern matches "World" in text regardless of case.

## ✓ 2. Special Characters

Special characters allow for more flexible searches.

### ✓ Dot (.)

The `.` character matches any single character except a newline. The dot helps create generic patterns, especially when you're unsure of certain characters.

```
pattern = "h.llo"
text = "hello"

match = re.search(pattern, text, re.IGNORECASE)
if match:
    print("Dot character match found:", match.group()) # Expected Output: "hello"
else:
    print("No match found")

➦ Dot character match found: hello
```

Here, the pattern "h.llo" matches any string with "h", followed by any character, then "llo" (e.g., "hello", "hallo").

### ✓ Caret (^)

The `^` character matches the start of a string. It is used to check if a string starts with a specific word or character.

```
pattern = "^hello"
print(re.search(pattern, "hello world"))
print(re.search(pattern, "world hello"))

➦ <re.Match object; span=(0, 5), match='hello'>
None
```

Here, `^hello` matches only if "hello" is at the start of the string.

### ✓ Dollar Sign (\$)

The `$` character matches the end of a string. Used to ensure that a string ends with a specific word or character.

```
pattern = "world$"
print(re.search(pattern, "hello world")) # Matches
print(re.search(pattern, "world hello")) # None

➦ <re.Match object; span=(6, 11), match='world'>
None
```

## ✓ 3. Quantifiers

Quantifiers specify how many times a character or group of characters can occur.

### ✓ Asterisk (\*)

- matches 0 or more occurrences of the preceding character. It allows flexible matching for any number of repetitions, including none.

```
pattern = "ab*c"
print(re.search(pattern, "ac"))
print(re.search(pattern, "abc"))
print(re.search(pattern, "abbc"))
```

```
➞ <re.Match object; span=(0, 2), match='ac'>
   <re.Match object; span=(0, 3), match='abc'>
   <re.Match object; span=(0, 4), match='abbc'>
```

ab\*c matches "ac", "abc", and "abbc" because \* allows for zero or more b's

#### ✓ Plus (+)

- matches 1 or more occurrences of the preceding character. To require at least one occurrence of a character.

```
pattern = "ab+c"
print(re.search(pattern, "ac"))      # None
print(re.search(pattern, "abc"))     # Matches
print(re.search(pattern, "abbc"))    # Matches
```

```
➞ None
   <re.Match object; span=(0, 3), match='abc'>
   <re.Match object; span=(0, 4), match='abbc'>
```

Here, ab+c requires at least one b after a to match. So "ac" does not match.

#### ✓ Question Mark (?)

? matches 0 or 1 occurrence of the preceding character. It allows optional matching, meaning the character may or may not be present.

```
pattern = "ab?c"
print(re.search(pattern, "ac"))      # Matches
print(re.search(pattern, "abc"))     # Matches
print(re.search(pattern, "abbc"))    # None
```

```
➞ <re.Match object; span=(0, 2), match='ac'>
   <re.Match object; span=(0, 3), match='abc'>
   None
```

ab?c matches both "ac" and "abc" but not "abbc" because b can appear at most once.

## ✓ Regex Functions

### ✓ 1. re.search()

The re.search() function searches for the first occurrence of a regex pattern within a string. If it finds a match, it returns a match object; otherwise, it returns None. Useful for checking if a particular substring or pattern exists within a larger string.

re.search(pattern, string) takes the pattern to search for and the string to search within.

```
import re

# Example
text = "Python programming is fun"
match = re.search(r"programming", text)
if match:
    print(f"Found at position: {match.start()}-{match.end()}")
else:
    print("Pattern not found")
```

```
➞ Found at position: 7-18
```

The re.search() function looks for "programming" in text. If it finds it, match.start() gives the start index, and match.end() gives the end index of the match.

## 2. re.match()

re.match() checks only at the beginning of a string to see if a pattern matches. When you need to verify if a string starts with a specific substring or pattern.

re.match(pattern, string) searches from the start of string.

```
print(re.match(r"Python", "Python programming")) # Matches
print(re.match(r"programming", "Python programming")) # None
```

```
>>> <re.Match object; span=(0, 6), match='Python'>
None
```

In the first line, re.match() finds "Python" at the start of the string and returns a match. In the second line, it doesn't find "programming" at the start, so it returns None.

## 3. re.findall()

re.findall() finds all non-overlapping matches of a pattern in a string and returns them as a list. Great for extracting all occurrences of a particular pattern in text (like email addresses, hashtags, etc.).

re.findall(pattern, string) returns a list of all matches found in string.

```
text = "Contact us at info@example.com or support@example.com"
emails = re.findall(r'\b[\w\.-]+@[\w\.-]+\.\w+\b', text)
print(emails)
```

```
>>> ['info@example.com', 'support@example.com']
```

The regex pattern \b[\w\.-]+@[\w\.-]+\.\w+\b matches email addresses. re.findall() returns all matches in a list: ['info@example.com', 'support@example.com'].

## 4. re.sub()

re.sub() replaces occurrences of a pattern with a specified replacement. Ideal for text cleaning or transformations, like replacing specific words or phrases.

re.sub(pattern, replacement, string) replaces all matches of pattern in string with replacement.

```
text = "I love cats, cats are cute"
new_text = re.sub(r"cats", "dogs", text)
print(new_text)
```

```
>>> I love dogs, dogs are cute
```

The pattern "cats" is replaced with "dogs" throughout the string. Output: "I love dogs, dogs are cute".

## 5. re.split()

re.split() splits a string by occurrences of a pattern. Useful for breaking down strings into components based on delimiters, such as splitting sentences, separating names, etc.

re.split(pattern, string) splits string at each occurrence of pattern.

```
text = "John,Paul;George:Ringo"
names = re.split(r'[,:;]', text)
print(names)
```

```
>>> ['John', 'Paul', 'George', 'Ringo']
```

re.split() uses [, :, and ; as delimiters and splits text into ['John', 'Paul', 'George', 'Ringo'].

## Practical Applications of Regex

## ✓ 1. Email Validation

Email validation using regex helps determine if a given email address is in the correct format (i.e., [username@domain.extension](#)). Validating email addresses is essential in applications where we collect user contact information, as it helps prevent incorrect or incomplete data entries.

We use a regex pattern to define the structure of a valid email. For this purpose, we look for:

- An alphanumeric username with optional special characters (.\_%+)
- An @ symbol followed by a valid domain name
- A domain extension of at least two characters

```
import re # Import the regular expressions module

def validate_email(email):
    # Define a regex pattern for a valid email
    pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
    # Check if the email matches the pattern
    return bool(re.match(pattern, email))

# List of test emails
emails = ["user@example.com", "invalid.email@", "user@x.le.ccn", "not-an-email"]

# Validate each email and print the result
for email in emails:
    print(f'{email}: {validate_email(email)}')
```

```
➞ user@example.com: True
   invalid.email@: False
   user@x.le.ccn: True
   not-an-email: False
```

- The pattern used is `^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`.
- `re.match()` checks if each email follows this format.
- The function `validate_email()` returns `True` if the email is valid, and `False` otherwise.

## ✓ Math Functions and Utilities

Python has built-in math functions and a `math` module for more advanced operations.

### ✓ Basic Math Functions

#### ✓ `abs(x)`: Absolute Value of x

Returns the absolute (non-negative) value of a number. Useful in scenarios where only the magnitude is needed, regardless of sign.

```
print(abs(-5))
```

```
➞ 5
```

#### ✓ `pow(x, y)`: x Raised to the Power of y

Computes x raised to the power y. Powers are common in many applications, such as growth calculations, physics formulas, etc.

```
print(pow(2, 3))
```

```
➞ 8
```

#### ✓ `min()` and `max()`: Minimum and Maximum Values in a List

`min()` finds the smallest, and `max()` finds the largest value in a collection of numbers. Handy for finding bounds in datasets or arrays.

```
numbers = [1, 5, 2, 8, 3]
print(min(numbers))
print(max(numbers))
```

 1  
8

## ✓ Advanced Math Operations

Python's `math` module provides more advanced functions.

### ✓ `math.sqrt(x)`: Square Root of x

Returns the square root of x. Common in geometry, physics, and statistics.

```
import math

# Square root
print(math.sqrt(16))
```

 4.0

### ✓ `math.exp(x)`: Exponential Function ( $e^x$ )

Returns the value of e (Euler's number,  $\sim 2.718$ ) raised to the power x. Exponentials are essential in growth models, decay functions, and more.

```
import math

# Exponential
print(math.exp(1))
```

 2.718281828459045

### ✓ `math.log(x, base)`: Logarithms

Computes the logarithm of x. If base is not specified, it returns the natural logarithm (base e). Logarithms are crucial in data transformation, financial calculations, and algorithms.

```
import math

# Logarithms
print(math.log(10)) #Natural Logarithm (ln)
print(math.log10(100)) #Logarithm Base 10
```

 2.302585092994046  
2.0

## ✓ Rounding and Randomization

### ✓ Rounding Functions

Rounding functions help to adjust numbers to the nearest integer or a specified decimal place. These functions are useful in data cleaning, formatting outputs, and situations where precision needs to be controlled.

#### ✓ `round()`: Rounds to the nearest integer or decimal place

Rounds a number to the nearest integer by default. If you provide a second argument, it rounds to that many decimal places. It's essential in rounding numbers to improve readability, manage precision in financial calculations, or format outputs.

```
print(round(3.14159, 2))
```

 3.14

Here, 3.14159 is rounded to two decimal places, giving 3.14.

#### ✓ `math.ceil()`: Rounds up to the nearest integer

It always rounds a number up to the next integer. Useful when you need the smallest integer that is greater than or equal to a given number.

```
print(math.ceil(3.1))
```

↔ 4

#### ✓ `math.floor()`: Rounds down to the nearest integer

It rounds a number down to the nearest integer. Often used when calculating indices or reducing values to a smaller, consistent integer.

```
print(math.floor(3.9))
```

↔ 3

#### ✓ Random Number Generation

Python's `random` module provides functions for generating random numbers and making random choices.

##### ✓ 1. `random.randint(a, b)`

Generates a random integer between `a` and `b` (both inclusive). Helpful for generating random IDs, simulating dice rolls, or creating random values in a given range.

```
import random  
print(random.randint(1, 10))
```

↔ 9

This function will output a random integer from 1 to 10, inclusive.

##### ✓ 2. `random.random()`

Generates a random floating-point number between 0 (inclusive) and 1 (exclusive). Useful in probability-based algorithms or when generating random fractions of values.

```
print(random.random())
```

↔ 0.8013447827342798

This function provides a decimal number that can be used in probability simulations or other applications requiring fractional random values.

##### ✓ 3. `random.uniform(a, b)`

Returns a random floating-point number between `a` and `b`. Useful for generating random weights, distances, or any continuous values within a specific range.

```
print(random.uniform(1, 10))
```

↔ 7.520287787636679

This will output a random decimal number within the range of 1 to 10.

##### ✓ 4. `random.choice(seq)`

Selects a random element from a non-empty sequence (like a list). Great for randomly selecting items from a list, such as quiz questions, products, or lottery draws.

```
fruits = ['apple', 'banana', 'orange']  
print(random.choice(fruits))
```

↔ orange



This function selects one random item from the fruits list.

✓ 5. `random.shuffle(seq)`

Shuffles a list in place, changing the order of elements randomly. Useful for shuffling decks of cards, creating random order in lists, or generating randomized playlists.

```
numbers = list(range(1, 6))  
random.shuffle(numbers)  
print(numbers)
```

↔ [1, 4, 2, 5, 3]

This function modifies numbers by rearranging its elements randomly.



# THANK YOU



**Stay updated with us!**



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)